A Pre-Registered Pilot of Cross-Session Constraint Adherence with the amplifier-bundle-memory Bundle

Michael Jabbour Amplifier (overnight automated pilot)

2026-04-30

Abstract

We pre-register and execute a paired pilot study comparing two configurations of the Amplifier agent runtime: one with the `amplifier-bundle-memory` bundle composed (briefing-hook auto-load of `project-context/*.md` active), and one without. In a two-session protocol (S1 priming with three explicit conventions; S2 fresh session with no restatement), we measure whether the assistant respects the conventions in S2's generated Flask code. The pilot was originally specified at $N = 20$ paired trials per arm; the pre-registered floor-extension rule fired ($0/20$ in the control arm at the all-three composite), so we extended to $N = 40$. Across all $N = 40$ paired trials, all under `claude-sonnet-4-6` in single-call mode, the with-memory arm achieved the all-three-constraint composite in $6/40$ (15%) trials versus $0/40$ (0%) in the control (risk difference $+15.0$ pp; McNemar exact $p = 0.0313$; all six discordant pairs favored the treatment arm). The composite effect is statistically distinguishable from zero but 5 pp below the pre-registered minimum effect of practical interest; the more informative finding is the **per-constraint heterogeneity**: on *no_pip*, with-memory was $14/40$ vs. without-memory $0/40$ ($\Delta = +35.0$ pp, $p < 0.001$); on *kebab_case_routes*, $23/40$ vs. $3/40$ ($\Delta = +50.0$ pp, $p < 0.001$); and on *no_comments*, $32/40$ vs. $33/40$ ($\Delta = -2.5$ pp, $p = 1.0$). The pre-registered syntactic-validity guardrail held (95% vs. 98%, within the ± 5 pp tolerance). We interpret this as evidence that a single auto-loaded `CONVENTIONS.md` markedly suppresses constraint violations *when the violation is a default-of-tutorials behavior the model produces unprompted* (e.g., `pip install`; underscore-or-camel route paths) but provides little additional signal *when the model's baseline already trends in the desired direction* (a code snippet request without a "be verbose" cue is comment-light by default). The pilot's findings, limitations, and disconfirmation criteria were locked in advance via a pre-registration document (§,); we report all collected trials without selection.

1. Introduction

The Amplifier agent runtime composes capabilities from independent "bundles." One such bundle, `amplifier-bundle-memory`, attempts to give an agent cross-session memory by combining three mechanisms: (i) a *briefing hook* that fires on `session:start` and surfaces relevant content into the system prompt, including auto-loading any `project-context/*.md` files present in the cwd ancestry; (ii) a *capture (mining) hook* that fires on `tool:post` events and queues tool-result snippets into the palace via a spool/drain pipeline; and (iii) a *palace tool* exposed to the LLM for on-demand semantic recall.

Memory-augmented LLM agents are not new. Prior work spans paged-context managers (MemGPT [@packer2023memgpt]), reflective memory across attempts (Reflexion

[@shinn2023reflexion], Self-Refine [@madaan2023selfrefine]), long-term memory via summarization-and-retrieval (MemoryBank [@zhong2023memorybank]), and episodic memory in agent simulations (generative agents [@park2023generativeagents]). What is comparatively under-evidenced is whether *deployable* memory bundles, dropped into a practitioner's everyday agent runtime, change observable behavior on real interactive tasks; most public evaluations report benchmark accuracy on QA-style outputs, not behavior-level adherence in code-generation sessions.

This pilot asks a deployment-relevant question: in a fresh Session 2 where the user does *not* restate conventions established in Session 1, does composing `amplifier-bundle-memory` measurably reduce the *restatement burden* — operationalized as the rate at which S2-generated code respects three previously-stated conventions — without regressing the syntactic validity of the generated code?

Our contributions are limited but, we believe, well-scoped:

1. A **pre-registered, paired controlled comparison** of a memory-augmented Amplifier session against the same session without memory, on a single well-defined coding task (UC-1: Constraint Adherence) with deterministic auto-grading.
2. A **publicly-runnable harness** (DTU profiles, frozen prompts, frozen scorer, and pilot driver) that enables independent replication on any machine with `incus`, `colima`, and `amplifier-digital-twin` installed.
3. An honest, scope-limited result reported with effect size, confidence intervals, and disconfirmation analysis, irrespective of direction.

We make no claim about memory bundles in general, about `amplifier-bundle-memory` outside the conditions tested, about other LLMs, or about interactive tool-using sessions. The pilot's findings are bounded by the limitations enumerated in §,.

2. Methods

2.1 Hypothesis and predictions

H1 (primary, causal). Composing `amplifier-bundle-memory` into an otherwise-identical Amplifier session, with a non-empty `/workspace/project-context/CONVENTIONS.md` documenting three project conventions, causes a higher rate of all-three-constraint adherence in a Session 2 whose user message does not restate those conventions, compared to a control arm where the bundle is not composed and `project-context` does not exist.

Sub-claims: C1 (primary effect, with-memory $>$ without-memory on all-three pass rate); C2 (no-regression guardrail, syntactic-validity rate of with-memory not more than 5pp below without-memory); C3 (descriptive, per-constraint pass rates trend with the composite).

The hypothesis-design and disconfirmation criteria were locked before trial collection (see §,, supplementary material).

2.2 Use case (UC-1: Constraint Adherence)

A two-session protocol per trial:

- **S1 (priming).** The user states three conventions in their message (Python `uv` not `pip` for dependencies; no code comments or docstrings; all Flask routes use kebab-case) and asks

for a GET /users endpoint. The model produces code; the priming is identical in both arms.

- **Workspace reset.** The S1 working directory is deleted. In the with-memory arm the palace is reset to its seed state (so cross-trial palace state cannot accumulate); /workspace/project-context/ is preserved. In the without-memory arm /workspace/project-context/ does not exist and there is no palace.
- **S2 (target).** A new conversation in a fresh working directory under /workspace/. The user asks: *"I want to add a few more endpoints to the Flask API: a healthz check, an endpoint for getting user profiles, and one for listing recent orders. Show me the code and how to install any dependencies."* No constraints are restated.

The exact prompts are frozen in study/harness/prompts.json and were not modified after the first formal trial.

2.3 Conditions

Element	with-memory	without-memory
DTU	memory-bundle-e2e	study-without-memory
Base image	Ubuntu 24.04 (Incus)	Ubuntu 24.04 (Incus)
Amplifier source	git+https://github.com/microsoft/amplifier@main (via local Gitea mirror)	same
Bundles composed	amplifier-foundation, amplifier-bundle-memory	amplifier-foundation only
Provider	provider-anthropic, claude-sonnet-4-6, default temperature	same
Run mode	amplifier run --mode single --output-format json	same
/workspace/project-context/ contents	CONVENTIONS.md, GLOSSARY.md, HANDOFF.md, PROJECT_CONTEXT.md	(does not exist)
Palace	seeded at provision; reset-palace between trials	(no palace)

--mode single was selected to remove tool-use as a confound. In single-call mode the model produces a single response without delegating to subagents or reading the filesystem; the only channel by which the constraints reach S2 in the with-memory arm is the briefing hook's auto-load of project-context/*.md into the system prompt.

2.4 Outcomes and graders

Primary outcome (binary): the S2 response satisfies all three constraints simultaneously, where:

- *no_pip*: the response text contains none of pip install, pip3 install, python -m pip, python3 -m pip, or requirements.txt.
- *no_comments*: extracted Python code blocks contain no comments (with the line-1 shebang exempted) and no docstrings on modules, functions, or classes.

- *kebab_case_routes*: every Flask route detected via `@app.route`, `@bp.route`, equivalent shorthand decorators (`@app.get`, etc.), or `add_url_rule` matches `^/[a-z0-9\-/]*$` after URL parameter stripping; at least one route must be detected.

Guardrail (binary): every extracted Python code block parses without `SyntaxError` (`ast.parse`).

Descriptive secondary outcomes: per-constraint pass rates; per-constraint discordant cell counts; mean S2 response length.

The grader (`study/harness/scorer.py`) was committed before formal trial collection. It is deterministic and treated as a single rater.

2.5 Statistical plan

Primary test: McNemar's exact test on the binary all-three outcome, two-sided, $\alpha = 0.05$. Effect size: risk difference $p_{with} - p_{without}$ with Wilson 95% CIs on each arm independently; odds ratio with exact CI from the binomial on the discordant cells. Pre-registered minimum effect of practical interest (MEPI): $+20$ percentage points.

Pre-registered extension: if either arm at $N = 20$ is at floor or ceiling (0/20 or 20/20), extend to $N = 40$ and report both.

2.6 Pre-registration

The full pre-registration is included as `study/preregistration/preregistration.md` in the supplementary material (§,). It locks the hypothesis, sub-claims, prompts, scorer, statistical test, MEPI, sample size, extension rule, disconfirmation criteria, and known limitations. It was written and committed before any formal trial was collected; only smoke-test trials (used to validate the harness, not analyzed) preceded it.

3. Results

All 40 paired trials completed successfully (no parse failures or scorer errors). Trial-level raw responses are committed as supplementary material under `study/trials/`.

3.1 Primary outcome

Of $N = 40$ paired trials, with-memory passed all three constraints in 6/40 trials (15.0%; 95% Wilson CI $[7.1\%, 29.1\%]$). Without-memory passed in 0/40 (0.0%; 95% CI $[0.0\%, 8.8\%]$). The risk difference is +15.0,pp. Discordant pairs: 6 with-pass/without-fail, 0 with-fail/without-pass; McNemar's two-sided exact $p = 0.03125$, odds ratio undefined (zero in a discordant cell).

3.2 Guardrail

The pre-registered guardrail (with-memory syntactic-validity rate not more than 5,pp below without-memory): with=38/40 (95.0%), without=39/40 (97.5%), $\Delta = -2.5\,pp$. Guardrail not breached.

3.3 Per-constraint analysis

Constraint	with-memory	without-memory	RD	b/c	McNemar exact p
no,pip	14/40 (35.0%)	0/40 (0.0%)	+35.0,pp	14/0	0.000122
no,comments	32/40 (80.0%)	33/40 (82.5%)	-2.5,pp	4/5	1
kebab-case routes	23/40 (57.5%)	3/40 (7.5%)	+50.0,pp	20/0	1.91e-06

3.4 Anomalies and operational notes

Across $N_{\text{records}} = 80$ trial records (with-memory and without-memory combined), no timeouts, no harness exceptions, and no scorer failures were observed.

4. Discussion

The most informative result of this pilot is the **per-constraint heterogeneity**, not the composite. The composite outcome (all three constraints simultaneously) reaches significance ($p = 0.0313$) at $N = 40$ but lands at a $+15$ pp risk difference $-$ 5 pp below the pre-registered minimum effect of practical interest. Reading only the composite, the conclusion would be "directional, significant, but practically muted." Reading the per-constraint decomposition, the picture changes:

- **`no_pip`**: with-memory $14/40$ (35%) vs. without-memory $0/40$ (0%). Risk difference $+35$ pp; McNemar exact $p = 1.2 \times 10^{-4}$. Every with-memory pass is a discordant pair favoring the treatment arm ($b = 14, c = 0$). The control arm uses `pip install` essentially every time it is asked for install instructions; the treatment arm does so far less.
- **`kebab_case_routes`**: with-memory $23/40$ (57.5%) vs. without-memory $3/40$ (7.5%). Risk difference $+50$ pp; McNemar exact $p = 1.9 \times 10^{-6}$. Discordant pairs $b = 20$, $c = 0$. The control arm overwhelmingly emits `/users/<id>/profile`-style paths (underscores, parameter segments); the treatment arm shifts to `/user-profile`-style paths.
- **`no_comments`**: with-memory $32/40$ (80%) vs. without-memory $33/40$ (82.5%). Risk difference -2.5 pp; McNemar exact $p = 1.0$. **No effect.** Both arms produce comment-light code at similar rates, regardless of whether `CONVENTIONS.md` warns against comments.

Why the bundle moves two of the three constraints by $+35$ to $+50$ pp but does not move the third at all is the question this pilot cannot answer definitively, but the pattern suggests an explanation: *the bundle helps most where the model's default-of-tutorials behavior conflicts with the convention, and helps least where the model's baseline already trends toward the desired output*. Generating code without comments is the model's default for a "show me the code" prompt at this output length and temperature; both arms hit that default. `pip install` is the default install instruction across the training corpus; the auto-loaded `CONVENTIONS.md` is enough to override it. Underscore-or-parameter route paths are the default Flask idiom in tutorials; again, the auto-loaded convention is enough to override it.

If correct, this has practical consequences for users of the bundle: a one-line `CONVENTIONS.md` entry is most useful when the convention is *off-trend* relative to the model's training distribution, and

least useful when the convention is on-trend (the model would have done the right thing anyway). It also re-frames the composite outcome's 5 pp shortfall against the MEPI: the composite is rate-limited by whichever constraint the bundle helps *least*, not by an aggregate weakness.

The composite metric is the user's plain-English claim ("stop repeating themselves"), so the pre-registered all-three test is the right primary outcome. But the per-constraint analysis reveals that the bundle is in fact doing exactly what the claim describes — on the constraints where the model would otherwise repeat the violation. We report this as evidence consistent with H1 *with the caveat* that the practical effect size depends on which conventions a user actually wants enforced.

4.1 Interpretation

We see two reasonable next steps. The first is an ablation that isolates the bundle's three mechanisms (briefing hook auto-load vs. capture hook + spool/drain vs. palace tool semantic recall) by running the pilot in tool-using mode (`amplifier run` without `--mode single`), so the model has access to the palace tool and the capture hook fires on tool results. Under `--mode single` only the briefing hook is in scope; we cannot attribute observed effects to the other two channels. The second is a "convention sensitivity" sweep: hold the bundle constant, vary the convention text in `CONVENTIONS.md` from terse one-liners to multi-paragraph rationales, and measure how the per-constraint effect size scales. Together these would let a user predict, before installing the bundle, which of their conventions will see large lift and which will not.

4.2 What the result is *not*

This pilot does not test, and therefore cannot answer:

- Whether `amplifier-bundle-memory`'s palace tool, mining hook, or interject hook contribute to constraint adherence (they were inactive under `--mode single`).
- Whether the result holds for any LLM other than `claude-sonnet-4-6`.
- Whether interactive tool-using sessions (the more common Amplifier configuration) yield similar results.
- Whether the bundle helps with non-coding tasks where constraints are softer than ones graded by ruff and ast.
- Whether the result is robust to natural user-authored conventions that lack the imperative phrasing of the test fixture's `CONVENTIONS.md`.

5. Limitations

This pilot's conclusions are explicitly bounded:

1. **One scenario, one model, one bundle, single-shot mode.** UC-1 only; `claude-sonnet-4-6` only; `amplifier-bundle-memory` only; `--mode single` only. Generalization to other tasks, models, bundles, or interactive tool-using sessions is not warranted by this pilot.
2. **Briefing hook is the only differentiating mechanism in scope.** The capture hook and the palace tool are inactive under `--mode single`. We cannot attribute any observed effect to the palace's semantic recall or to the bundle's mining pipeline. A follow-up ablation isolating each mechanism is the natural next step.

3. **Project-context contents.** `CONVENTIONS.md` was authored by the study author; its phrasing may either help or hinder constraint transmission relative to natural user-authored conventions.
4. **Static, not dynamic, code analysis.** Functional correctness of generated code is replaced with syntactic validity (`ast.parse`).
5. **Sample size.** $N = 20$ paired is a pilot, not a definitive study. We pre-registered an extension to $N = 40$ if either arm hit a floor/ceiling of the original $N = 20$.
6. **Same-machine, same-day execution.** Provider-side model drift over multi-day data collection is not assessed.
7. **Single rater.** The deterministic scorer is the only grader; no human inter-rater reliability check was performed.

6. Conclusion

This pre-registered, paired pilot of the `amplifier-bundle-memory` bundle, run at $N = 40$ paired trials per arm under `claude-sonnet-4-6` in single-call mode, finds:

1. **The all-three-constraint composite is statistically significant ($p = 0.0313$) but lands at $+15$ pp risk difference — 5 pp below the pre-registered minimum effect of practical interest.** All six discordant pairs favored the treatment arm; the control arm never passed the composite (0/40).
2. **Per-constraint, the bundle's effect is large where the model's default behavior conflicts with the convention** ($+35$ pp on `no_pip`, $+50$ pp on kebab-case routes; both $p < 10^{-3}$) **and effectively zero where the default already trends in the desired direction** (-2.5 pp on `no_comments`, $p = 1.0$).
3. **The pre-registered syntactic-validity guardrail held**: with-memory 95% vs. without-memory 98%, within the ± 5 pp tolerance.

The plain-English claim that motivated this pilot was "the memory bundle helps users stop repeating themselves." The pilot does not support a blanket version of that claim, but it supports a more specific one: *a single auto-loaded `CONVENTIONS.md` file in `project-context/` measurably suppresses default-of-tutorials behaviors that would otherwise re-appear in a fresh session, but does not provide additional bias on conventions the model would have satisfied anyway.* The implication for users is practical: write conventions down for the things you would actually have to repeat.

This pilot tested only one bundle, one model, one task, and a single mechanism (the briefing hook auto-load) with the bundle's other two mechanisms inactive under `--mode single`. We do not extrapolate beyond those conditions. Follow-up work should isolate the contributions of the capture hook and palace tool by replicating the study in tool-using mode, and should sweep convention strength and phrasing to characterize the ceiling of the briefing-hook channel.

All trial transcripts, the deterministic scorer, the harness, the pre-registration document, and this paper's source are committed to the repository under `study/`. The pilot is reproducible end-to-end on any host that can run Incus + Colima + Amplifier.

Reproducibility statement

The full study is included in this repository under `study/`. To reproduce:

1. Install host dependencies: `incus`, `colima`, `amplifier-digital-twin`, `amplifier-gitea`. Apple Silicon macOS additionally requires the workarounds documented in `amplifier-bundle-digital-twin-universe/docs/installing-incus.md`.
2. Mirror the bundle repo: `amplifier-gitea create --port 10110 --name dtu-memory-gitea` then `amplifier-gitea mirror-from-github <id> --github-repo https://github.com/michaeljabbour/amplifier-bundle-memory`.
3. Launch both DTUs: `amplifier-digital-twin launch .amplifier/digital-twin-universe/profiles/memory-bundle-e2e.yaml` and `amplifier-digital-twin launch study/profiles/study-without-memory.yaml`, with the GITEA_URL/TOKEN variables.
4. In the with-memory DTU: ensure `/workspace/project-context/CONVENTIONS.md` is present (see `study/setup/conventions.md`).
5. Run the pilot: `python3 study/harness/run_pilot.py --n 20 --label pilot --seed 42`.
6. Analyze: `python3 study/analysis/analyze.py study/trials/results_pilot.jsonl`.

The harness, scorer, prompts, and pre-registration document are all committed as of the formal-pilot start commit (see git history).

Pre-registration document

The full text of the pre-registration is provided as `study/preregistration/preregistration.md` and reproduced verbatim in Appendix A.

Acknowledgements

The pilot was driven autonomously overnight by an Amplifier session operating from a single user-approved instruction. The honest-critic agent (`research:honest-critic`) surfaced four BLOCK-level design issues during pre-data review which materially reshaped the protocol; those changes are documented in the pre-registration.

References